# DASH-04: Operations Dashboards

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 4 of 7 | **Created:** March 2026 | **Last Updated:** 10/14/2026

## Overview

Operations dashboards serve as the real-time control panel for SREs, platform engineers, and NOC teams. Unlike executive dashboards that summarize weekly trends, operations dashboards focus on what is happening right now — service response times, error rates, log volume anomalies, active problems, and infrastructure health. This notebook covers tile selection, query patterns for real-time monitoring, and layout strategies for operations-tier dashboards.

---

## Table of Contents


1. Service Response Time Monitoring
2. Error Rate by Service
3. Log Volume and Anomaly Tracking
4. Active Problems and Alerts
5. Infrastructure Health
6. Deployment Correlation
7. Operations Dashboard Layout
8. Summary and Next Steps


---

## Prerequisites

| Requirement | Details |
| --- | --- |
| **Dynatrace Environment** | Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards |
| **Permissions** | `storage:logs:read`, `storage:metrics:read`, `storage:events:read`, `storage:spans:read` |
| **Data** | Active services with span data, log ingestion, host metrics |
| **Prior Reading** | DASH-01, DASH-02 |

# 1. Service Response Time Monitoring

Response time is the most watched metric on an operations dashboard. Show it as a timeseries line chart with p50, p90, and p95 lines for context.

### Response Time Trend (All Services)

```
// Service response time trend — p50, p90, p95 over last 2 hours
//
// Corrected 08/12/2026: `makeTimeseries` takes a BARE aggregation — dividing inside it
// (`percentile(duration, 50) / 1ms`) fails with "the parameter has to be an expression-based
// timeseries aggregation". Aggregate first, convert afterwards in `fieldsAdd`. Note the unit
// conversion also changes: makeTimeseries yields a numeric array in NANOSECONDS, not a duration,
// so `/ 1ms` silently yields null on the array — divide by 1000000.0 instead.
fetch spans, from:-2h
| filter span.kind == "server"
| makeTimeseries p50 = percentile(duration, 50), p90 = percentile(duration, 90), p95 = percentile(duration, 95), interval:5m
| fieldsAdd p50_ms = p50[] / 1000000.0, p90_ms = p90[] / 1000000.0, p95_ms = p95[] / 1000000.0
| fieldsRemove p50, p90, p95
```

### Response Time by Service (Top 10 Slowest)

```
// Top 10 slowest services by average response time
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {avg_ms = avg(duration) / 1ms, p95_ms = percentile(duration, 95) /
1ms, request_count = count()}, by:{dt.entity.service}
| sort p95_ms desc
| limit 10
```

## 2. Error Rate by Service

Error rate as a percentage shows which services are degraded. Display as a bar chart sorted by error rate descending.

### Current Error Rate Table

```
// Error rate by service — operations bar chart
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {total = count(), errors = countIf(span.status_code == "error")},
by:{dt.entity.service}
| fieldsAdd error_rate_pct = round(100.0 * errors / total, decimals: 2)
| sort error_rate_pct desc
| limit 15
```

### Error Rate Trend Over Time

```
// Overall error rate trend — operations line chart
// Divide the arrays element by element so each 5-minute bucket gets its own
rate;
// arrayAvg(errors) / arrayAvg(total) would collapse the trend to a single
number.
// coalesce() turns buckets with no errors (null) into 0 instead of a gap.
fetch spans, from:-2h
| filter span.kind == "server"
| makeTimeseries {total = count(), errors = countIf(span.status_code ==
"error")}, interval:5m
| fieldsAdd error_rate = 100.0 * coalesce(errors[], 0) / total[]
| fieldsKeep timeframe, interval, error_rate
```

# 3. Log Volume and Anomaly Tracking

Sudden log volume changes often indicate problems before Dynatrace Intelligence detects them. Track log volume by level and source.

## Log Volume by Severity Level

```
// Log volume by severity — stacked area chart
fetch logs, from:-2h
| filterOut loglevel == "NONE"
| makeTimeseries log_count = count(), interval:5m, by:{loglevel}
```

## Top Log Sources by Error Volume

```
// Top 10 log sources generating errors — operations table
fetch logs, from:-1h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR"
| summarize error_count = count(), by:{log.source}
| sort error_count desc
| limit 10
```

## Log Volume Spike Detection

Compare current hour to the same hour yesterday to detect anomalous volume increases.

```
// Log volume comparison: current hour vs same hour yesterday
fetch logs, from:-1h
| summarize current_count = count()
| append [
    fetch logs, from:now() - 25h, to:now() - 24h
    | summarize yesterday_count = count()
  ]
```

# 4. Active Problems and Alerts

Active problems should be front and center on every operations dashboard. Show both the count and the detail.

### Active Problems List

```
// Active problems detail table — operations dashboard
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| filter dt.davis.is_duplicate == false
| fieldsAdd duration_min = (now() - event.start) / 1m
| fieldsKeep display_id, event.name, event.category, duration_min
| sort duration_min desc
```

### Problem Status Over Time

```
// Concurrently open problems over time — area chart
fetch dt.davis.problems, from:-24h
| makeTimeseries concurrent = count(), spread:timeframe(from:event.start,
to:coalesce(event.end, now()))
```

# 5. Infrastructure Health

Infrastructure tiles provide context for service-level issues. CPU, memory, and disk metrics help operations teams determine if a problem is service-specific or infrastructure-wide.

### Host CPU Usage

```
// Host CPU usage — operations line chart
timeseries avg_cpu = avg(dt.host.cpu.usage), from:-2h, by:{dt.entity.host}
```

### Top Hosts by CPU — Identifying Hot Spots

```
// Top 5 hosts by average CPU usage
timeseries avg_cpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avg_cpu_val = arrayAvg(avg_cpu)
| sort avg_cpu_val desc
| limit 5
```

### Host Memory Usage

```
// Host memory usage — operations line chart
timeseries avg_mem = avg(dt.host.memory.usage), from:-2h, by:{dt.entity.host}
```

## 6. Deployment Correlation

Deployments are the most common cause of production issues. Overlaying deployment events on service metrics helps operations teams immediately correlate "what changed."

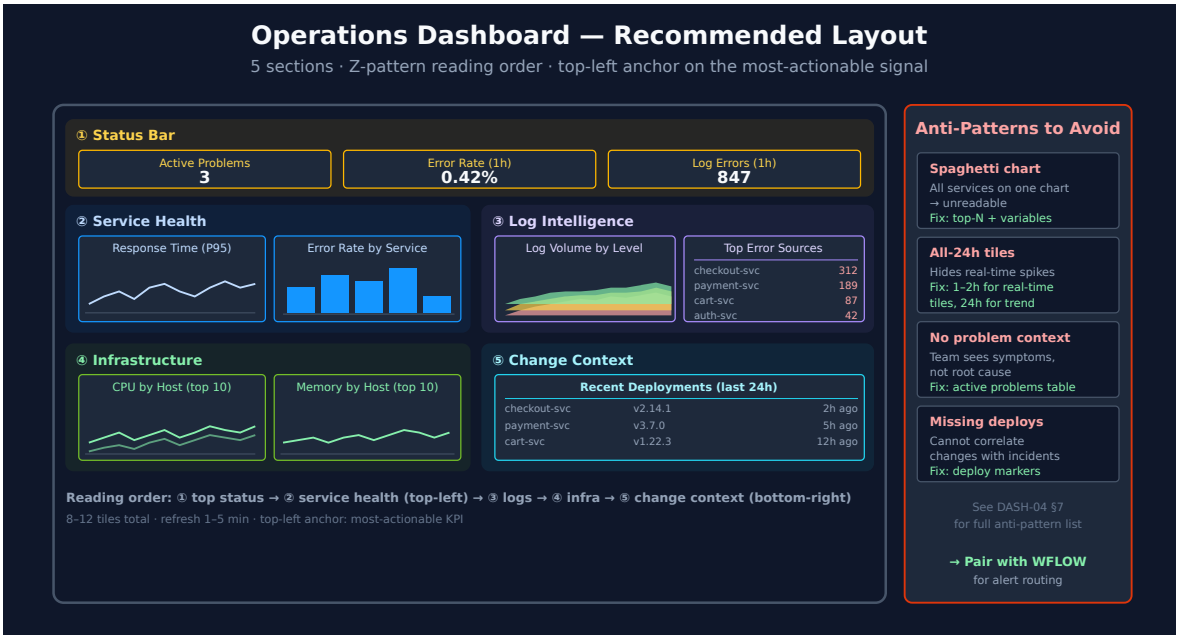
### Recent Deployment Events

Deployment events arrive in two shapes: classic Events-API deployments ( `event.type == "CUSTOM_DEPLOYMENT"` ) and SDLC deployment task events, where `event.kind` is always `SDLC_EVENT` , `event.category` is `task` and `event.type` is `deployment` . The query below catches both. The SDLC branch follows the semantic dictionary's model; it executes, but the validation tenant (09/28/2026) held no deployment events of either shape over 365 days, so it has not been checked against live SDLC data — confirm it returns your pipeline's deployments before putting it on a dashboard.

> **Sources:** SDLC events (DT docs) — `event.kind` : *"Automatically and unconditionally set to value `SDLC_EVENT` ."* **Dictionary:** models `sdlc.task.deployment.started.event` and `sdlc.task.deployment.finished.event` ( `data_object` `events` ), read 09/28/2026; no model uses the kind `DEPLOYMENT_EVENT` that an earlier version of this cell filtered on.

```
// Recent deployment events — classic (Events API) and SDLC deployment events
fetch events, from:-6h
| filter event.type == "CUSTOM_DEPLOYMENT"
    or (event.kind == "SDLC_EVENT" and event.category == "task" and event.type
== "deployment")
| fieldsKeep timestamp, event.kind, event.type, event.status, event.name,
dt.entity.service, cicd.deployment.name, cicd.deployment.release_stage
| sort timestamp desc
| limit 20
```

# 7. Operations Dashboard Layout




## Recommended Tile Arrangement

| Section | Tiles | Chart Type |
|---|---|---|
| **Status Bar** (top) | Active problems count, overall error rate, log error count | Single values |
| **Service Health** | Response time trend, error rate by service | Line chart, bar chart |
| **Log Intelligence** | Log volume by level, top error sources | Stacked area, table |
| **Infrastructure** | CPU by host, memory by host | Line charts |

| Section | Tiles | Chart Type |
|---|---|---|
| **Change Context** | Recent deployments | Table |

### Operations Dashboard Anti-Patterns

| Anti-Pattern | Problem | Fix |
|---|---|---|
| Too many services on one chart | Unreadable spaghetti lines | Use variables or top-N filtering |
| All tiles show 24h data | Hides real-time spikes | Use 1-2h for real-time tiles |
| No problem context | Team sees metrics but not root cause | Add active problems table |
| Missing deployment markers | Cannot correlate changes with incidents | Add deployment events section |

## 8. Summary and Next Steps

In this notebook you learned:

- How to build service response time and error rate tiles for real-time monitoring
- Log volume tracking and spike detection queries
- Active problem tables with duration tracking
- Infrastructure health tiles for CPU and memory
- Deployment event tracking for change correlation
- Recommended layout and common anti-patterns for operations dashboards

**Next:** In **DASH-05: Engineering Dashboards**, we move to the deepest tier — trace-level analysis, database performance, endpoint metrics, and before/after deployment comparisons.

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*